MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 33 — Métaprogrammation

Métaclasses, descripteurs, introspection — le code qui écrit du code

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

31 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 33 — Métaprogrammation **2**
0.1 Introspection — Explorer les objets 2
0.2 Les descripteurs — Contrôler l'accès aux attributs 3
0.3 Les métaclasses — Fabriquer des classes 5
0.4 __init_subclass__ — Alternative moderne aux métaclasses 8
0.5 contextlib — Gestionnaires de contexte avancés 9
0.6 Décorateurs de classe et $_{class_getitem}$ 11
0.7 Ce qui m'a le plus surpris dans ce chapitre 12
0.8 Résumé du Chapitre 33 13

Chapitre 33 — Métaprogrammation

À retenir

La **métaprogrammation** est l'art d'écrire du code qui manipule d'autres codes — des classes, des fonctions, des attributs — à l'exécution. En Python, tout est objet : les fonctions, les classes, même les types. Cette uniformité permet une introspection et une modification dynamique exceptionnelles. Ce chapitre couvre l'**introspection**, les **descripteurs**, les **métaclasses** et les **décorateurs de classe** — les mécanismes qui alimentent des frameworks comme Django, SQLAlchemy et FastAPI.

Introspection — Explorer les objets

Inspecter les objets à l'exécution

```python
import inspect
from typing import Any

class Etudiant:
    """Represente un etudiant."""
    NB_ETUDIANTS: int = 0

    def __init__(self, prenom: str, note: float):
        self.prenom = prenom
        self.note   = note
        Etudiant.NB_ETUDIANTS += 1

    def mention(self) -> str:
        """Retourne la mention de l'etudiant."""
        if self.note >= 16: return "Tres Bien"
        if self.note >= 10: return "Passable+"
        return "Echec"

    @classmethod
    def creer(cls, data: dict) -> "Etudiant":
        return cls(data["prenom"], data["note"])

alice = Etudiant("Alice", 15.5)

# Fonctions d'introspection fondamentales
print(type(alice))              # <class '__main__.Etudiant'>
print(type(Etudiant))           # <class 'type'>
print(isinstance(alice, Etudiant))   # True
print(issubclass(Etudiant, object))  # True
```

```python
# dir()      tous les attributs et methodes
attributs = [a for a in dir(alice)
             if not a.startswith("_")]
print(f"Attributs publics : {attributs}")

# vars() / __dict__      attributs d'instance
print(vars(alice))
# {'prenom': 'Alice', 'note': 15.5}

# getattr, setattr, hasattr, delattr
print(getattr(alice, "prenom"))            # Alice
print(getattr(alice, "age", "Inconnu"))    # Inconnu (defaut)

setattr(alice, "ville", "Abidjan")
print(alice.ville)     # Abidjan

print(hasattr(alice, "mention"))     # True
print(hasattr(alice, "voler"))       # False

# inspect      introspection avancee
print(inspect.isclass(Etudiant))        # True
print(inspect.ismethod(alice.mention)) # True
print(inspect.isfunction(Etudiant.mention)) # True

# Signature d'une fonction
sig = inspect.signature(Etudiant.__init__)
print(f"Parametres : {list(sig.parameters.keys())}")

# Code source
print(inspect.getsource(Etudiant.mention))

# Membres d'une classe
for nom, obj in inspect.getmembers(Etudiant):
    if not nom.startswith("_"):
        print(f"  {nom:20} : {type(obj).__name__}")
```

Les descripteurs — Contrôler l'accès aux attributs

Descripteurs — $_{get}, _{set}, _{delete}$

```python
class Valide:
    """Descripteur qui valide une valeur numerique bornee."""

    def __init__(self, minimum: float, maximum: float,
                 nom: str = ""):
        self.minimum = minimum
        self.maximum = maximum
        self.nom     = nom
```

```python

    def __set_name__(self, owner, name):
        """Appele quand le descripteur est assigne a une classe."""
        self.nom = name
        self.attribut_prive = f"_{name}"

    def __get__(self, instance, owner):
        """Appele lors de la lecture : obj.attribut"""
        if instance is None:
            return self    # acces depuis la classe
        return getattr(instance, self.attribut_prive, None)

    def __set__(self, instance, valeur):
        """Appele lors de l'ecriture : obj.attribut = val"""
        if not isinstance(valeur, (int, float)):
            raise TypeError(
                f"{self.nom} doit  tre  numerique, "
                f"re u {type(valeur).__name__}"
            )
        if not self.minimum <= valeur <= self.maximum:
            raise ValueError(
                f"{self.nom} doit  tre  entre "
                f"{self.minimum} et {self.maximum}, "
                f"re u {valeur}"
            )
        setattr(instance, self.attribut_prive, valeur)

    def __delete__(self, instance):
        """Appel  lors de del obj.attribut"""
        delattr(instance, self.attribut_prive)


class Etudiant:
    # Les descripteurs remplacent @property pour la reutilisation
    note    = Valide(0, 20, "note")
    age     = Valide(16, 99, "age")
    salaire = Valide(0, 1_000_000, "salaire")

    def __init__(self, prenom: str, note: float, age: int):
        self.prenom = prenom
        self.note   = note    # passe par __set__
        self.age    = age     # passe par __set__

    def __repr__(self):
        return (f"Etudiant(prenom={self.prenom!r}, "
                f"note={self.note}, age={self.age})")


```

```python
alice = Etudiant("Alice", 15.5, 22)
print(alice)
print(alice.note)    # passe par __get__

alice.note = 18.0    # OK
print(alice)

try:
    alice.note = 25    # ValueError !
except ValueError as e:
    print(f"Erreur : {e}")

try:
    alice.age = "vingt"    # TypeError !
except TypeError as e:
    print(f"Erreur : {e}")
```

Les métaclasses — Fabriquer des classes

Métaclasses — type est la métaclasse de tout

```python
# En Python, TOUT est un objet     y compris les classes
# La metaclasse d'une classe est son "type de type"

class MaClasse:
    pass

print(type(MaClasse))       # <class 'type'>
print(type(int))            # <class 'type'>
print(type(type))           # <class 'type'>  (type est sa
   propre meclasse)

# Creer une classe dynamiquement avec type()
# type(nom, bases, attributs)
Dynamique = type(
    "Dynamique",            # nom
    (object,),              # classes parentes
    {                       # attributs et methodes
        "x": 42,
        "saluer": lambda self: f"Bonjour depuis {self.__class__
           .__name__}"
    }
)

obj = Dynamique()
print(obj.x)            # 42
print(obj.saluer())     # Bonjour depuis Dynamique
```

```python
# Metaclasse personnalisee
class MetaValidation(type):
    """Verifie que toutes les methodes ont une docstring."""

    def __new__(mcs, nom, bases, namespace):
        # Inspecter les m thodes avant la cr ation
        for cle, valeur in namespace.items():
            if callable(valeur) and not cle.startswith("_"):
                if not valeur.__doc__:
                    raise TypeError(
                        f"La m thode '{cle}' de '{nom}' "
                        f"doit avoir une docstring !"
                    )
        cls = super().__new__(mcs, nom, bases, namespace)
        print(f"Classe '{nom}' creee avec MetaValidation")
        return cls

class ServiceBien(metaclass=MetaValidation):
    def calculer(self) -> float:
        """Calcule et retourne un resultat."""
        return 42.0

    def traiter(self) -> None:
        """Traite les donn es d'entree."""
        pass

try:
    class ServiceMal(metaclass=MetaValidation):
        def calculer(self):
            return 42    # PAS DE DOCSTRING !
except TypeError as e:
    print(f"Erreur : {e}")
```

Métaclasse — cas pratique registre automatique

```python
class MetaRegistre(type):
    """Enregistre automatiquement toutes les sous-classes."""
    _registre: dict = {}

    def __new__(mcs, nom, bases, namespace):
        cls = super().__new__(mcs, nom, bases, namespace)
        if bases:    # ne pas enregistrer la classe de base
            MetaRegistre._registre[nom] = cls
            print(f"      Enregistre : {nom}")
        return cls

    @classmethod
    def obtenir(mcs, nom: str):
        return mcs._registre.get(nom)
```

```python

    @classmethod
    def tous(mcs) -> dict:
        return dict(mcs._registre)


class Forme(metaclass=MetaRegistre):
    """Classe de base des formes geometriques."""
    def aire(self) -> float:
        raise NotImplementedError

class Cercle(Forme):
    """Cercle."""
    def __init__(self, r): self.r = r
    def aire(self): return 3.14159 * self.r**2

class Rectangle(Forme):
    """Rectangle."""
    def __init__(self, l, L): self.l, self.L = l, L
    def aire(self): return self.l * self.L

class Triangle(Forme):
    """Triangle."""
    def __init__(self, b, h): self.b, self.h = b, h
    def aire(self): return 0.5 * self.b * self.h

print(f"\nFormes enregistr es : {list(MetaRegistre.tous())}")

# Instancier depuis le registre     factory pattern
def creer_forme(nom: str, *args) -> Forme:
    cls = MetaRegistre.obtenir(nom)
    if cls is None:
        raise ValueError(f"Forme inconnue : {nom}")
    return cls(*args)

cercle    = creer_forme("Cercle", 5)
rectangle = creer_forme("Rectangle", 3, 4)
print(f"\nCercle    aire : {cercle.aire():.2f}")
print(f"Rectangle aire : {rectangle.aire():.2f}")
```

__init_subclass__ — Alternative moderne aux métaclasses

init_subclass_contrôlerl'héritage

```python
class Plugin:
    """Systeme de plugins avec __init_subclass__."""
    _plugins: dict = {}

    def __init_subclass__(cls, nom_plugin: str = "",
                          version: str = "1.0",
                          **kwargs):
        """Appel  automatiquement a chaque sous-classe."""
        super().__init_subclass__(**kwargs)
        if not nom_plugin:
            nom_plugin = cls.__name__.lower()
        Plugin._plugins[nom_plugin] = {
            "classe":  cls,
            "version": version
        }
        print(f"Plugin '{nom_plugin}' v{version} enregistre.")

    @classmethod
    def obtenir(cls, nom: str):
        info = cls._plugins.get(nom)
        return info["classe"] if info else None

    @classmethod
    def lister(cls) -> list:
        return list(cls._plugins.keys())


class PluginCSV(Plugin, nom_plugin="csv", version="2.1"):
    def traiter(self, fichier: str) -> dict:
        """Traite un fichier CSV."""
        return {"fichier": fichier, "format": "csv"}

class PluginJSON(Plugin, nom_plugin="json", version="1.5"):
    def traiter(self, fichier: str) -> dict:
        """Traite un fichier JSON."""
        return {"fichier": fichier, "format": "json"}

class PluginXML(Plugin, nom_plugin="xml"):
    def traiter(self, fichier: str) -> dict:
        """Traite un fichier XML."""
        return {"fichier": fichier, "format": "xml"}

print(f"\nPlugins disponibles : {Plugin.lister()}")

cls_csv = Plugin.obtenir("csv")
plugin  = cls_csv()
```

```python
print(plugin.traiter("donnees.csv"))
```

contextlib — Gestionnaires de contexte avancés

contextlib — outils pour with

```python
from contextlib import (contextmanager, suppress,
                        redirect_stdout, redirect_stderr,
                        ExitStack)
import io

# contextmanager     gestionnaire en generateur
@contextmanager
def transaction_db(connexion):
    """Gestionnaire de transaction."""
    print("BEGIN TRANSACTION")
    try:
        yield connexion
        print("COMMIT")
        connexion["committed"] = True
    except Exception as e:
        print(f"ROLLBACK : {e}")
        connexion["committed"] = False
        raise

conn = {"committed": False, "data": []}
try:
    with transaction_db(conn) as c:
        c["data"].append("INSERT INTO ...")
        c["data"].append("UPDATE ...")
        # raise ValueError("Erreur simul e")
except:
    pass
print(f"Commit : {conn['committed']}")

# suppress     ignorer des exceptions
with suppress(FileNotFoundError, PermissionError):
    import os
    os.remove("fichier_inexistant.txt")
    print("Supprim ")
# Pas d'erreur levee meme si le fichier n'existe pas

# redirect_stdout     capturer la sortie
sortie = io.StringIO()
with redirect_stdout(sortie):
    print("Ceci va dans la capture")
    print("Et ceci aussi")
texte_capture = sortie.getvalue()
```

```python
print(f"Capture : {texte_capture!r}")

# ExitStack      gestionnaires de contexte dynamiques
noms_fichiers = ["f1.txt", "f2.txt", "f3.txt"]
with ExitStack() as stack:
    # Ouvrir N fichiers dynamiquement
    fichiers = [
        stack.enter_context(
            open(f, "w", encoding="utf-8")
        )
        for f in noms_fichiers
    ]
    for i, f in enumerate(fichiers):
        f.write(f"Contenu {i+1}\n")
# Tous les fichiers sont ferm s automatiquement

# Gestionnaire de contexte comme classe
class Minuteur:
    """Mesure le temps d'un bloc with."""
    def __init__(self, nom: str = ""):
        self.nom     = nom
        self.duree   = 0.0

    def __enter__(self):
        import time
        self._debut = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        self.duree = time.perf_counter() - self._debut
        print(f"[{self.nom}] {self.duree*1000:.2f} ms")
        return False    # ne pas supprimer les exceptions

with Minuteur("Tri d'un million") as m:
    import random
    data = random.sample(range(1_000_000), 1_000_000)
    data.sort()
print(f"Dur e enregistree : {m.duree:.4f}s")
```

Décorateurs de classe et $_{class_getitem}$

Décorateurs de classe et paramétrage

```python
from typing import Type, TypeVar
import functools

T = TypeVar("T")

# Decorateur de classe     modifier une classe entiere
def singleton(cls: Type[T]) -> Type[T]:
    """Transforme une classe en singleton."""
    instances = {}

    @functools.wraps(cls)
    def obtenir_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]

    return obtenir_instance

@singleton
class Config:
    def __init__(self):
        self.debug = False
        self.port  = 8000
        print("Config instanciee.")

c1 = Config()
c2 = Config()
print(c1 is c2)   # True     m me objet

# Decorateur de classe     ajouter des methodes
def avec_repr(cls: Type[T]) -> Type[T]:
    """Ajoute un __repr__ automatique depuis les annotations."""
    def __repr__(self) -> str:
        champs = []
        for attr, _ in cls.__annotations__.items():
            val = getattr(self, attr, "?")
            champs.append(f"{attr}={val!r}")
        return f"{cls.__name__}({', '.join(champs)})"
    cls.__repr__ = __repr__
    return cls

def avec_eq(cls: Type[T]) -> Type[T]:
    """Ajoute un __eq__ automatique depuis les annotations."""
    def __eq__(self, other) -> bool:
```

```python
        if type(self) != type(other):
            return False
        return all(
            getattr(self, a) == getattr(other, a)
            for a in cls.__annotations__
        )
    cls.__eq__ = __eq__
    return cls

@avec_repr
@avec_eq
class Point:
    x: float
    y: float

    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

p1 = Point(1.0, 2.0)
p2 = Point(1.0, 2.0)
p3 = Point(3.0, 4.0)

print(p1)            # Point(x=1.0, y=2.0)
print(p1 == p2)      # True
print(p1 == p3)      # False
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `type` est la métaclasse de tout — y compris de lui-même. `type(int)` retourne `type`. `type(type)` retourne aussi `type`. Cette auto-référence m'a semblé être un paradoxe logique. En réalité, c'est le fondement de l'uniformité de Python — tout objet a un type, et ce type est une classe. Les classes sont des objets de type `type`. C'est récursif, cohérent, et une fois compris, on réalise que Python est construit sur un principe unique appliqué partout.

2. Les descripteurs alimentent tout Python. `@property`, `@staticmethod`, `@classmethod` — ce sont tous des descripteurs. Les fonctions elles-mêmes sont des descripteurs (c'est pourquoi `obj.methode()` passe automatiquement `self`). Comprendre les descripteurs, c'est comprendre comment Python résout l'accès aux attributs à chaque niveau de la hiérarchie MRO. C'est le mécanisme le plus fondamental et le moins enseigné de Python.

3. `__init_subclass__` remplace les métaclasses dans 90% des cas. Les métaclasses sont puissantes mais complexes. `__init_subclass__` fait la même chose

pour le cas d'usage le plus fréquent — réagir à la création d'une sous-classe — avec une syntaxe infiniment plus simple. Django, FastAPI et SQLAlchemy utilisent ces techniques pour construire leurs systèmes de modèles, de routes et de schémas. Comprendre la métaprogrammation, c'est comprendre comment les frameworks fonctionnent vraiment.

Résumé du Chapitre 33

Ce que j'ai appris dans ce chapitre :

- **Introspection** : `type()`, `dir()`, `vars()`, `inspect`.
- `getattr/setattr/hasattr/delattr` — manipuler les attributs dynamiquement.
- **Descripteurs** : `__get__`, `__set__`, `__set_name__` — réutiliser `@property`.
- **Métaclasses** : `type` comme fabrique de classes, `__new__`.
- `type(nom, bases, namespace)` — créer des classes dynamiquement.
- `__init_subclass__` — alternative moderne aux métaclasses.
- `contextlib.suppress` — ignorer des exceptions proprement.
- `contextlib.ExitStack` — gestionnaires de contexte dynamiques.
- **Décorateurs de classe** — modifier des classes entières.

"La métaprogrammation n'est pas de la magie. C'est Python qui applique ses propres règles à lui-même. Quand on comprend le modèle objet de Python, tout devient logique — même l'extraordinaire."